

ESM Credit LOS — End-to-End Business Flows

Traced from real code (backend/src/credit/ routes → controllers → services → Prisma). All URL patterns are mounted under /api/v1/credit.

Global gates on every flow (credit.routes.ts:202-203): authenticate (JWT) + requireFeatureFlag('credit:module') (module toggle, cached 60s). Route-level permission tiers: credit:read / write / approve / disburse / admin.

Shared cross-cutting middleware:

- **RM scope** applyRmScope() (middleware/rmScope.middleware.ts) — non-admin users see only apps they are RM/analyst on (rmScopeFilter = { OR: [{assignedRmId},{assignedAnalystId}] }). Bypass roles: ADMIN, CREDIT_ADMIN, CREDIT_MANAGER, or credit:admin permission.
- **Application access** requireApplicationAccess() (applicationAccess.middleware.ts) — mounted once at router.use('/applications/:applicationId', applyRmScope(), requireApplicationAccess()) (credit.routes.ts:242) so **all** nested app routers inherit it. Out-of-scope direct-ID reads → **404** (not 403, to avoid existence disclosure).
- **SOD** enforceCreditSOD() / enforceCommitteesSOD() (middleware/sod.middleware.ts) — (1) assigned RM can't approve own app, (2) maker-checker (last state-transition/submission actor can't be approver), (3) admin bypass logs SOD_BYPASSED.
- **MFA** requireMfa (middleware/mfa.middleware.ts) — fresh (≤15 min) MFA required for paths matching /approvals, /disburse, /approve, /disbursement; also feature-flag mutations.
- **Audit chain** AuditChainService (services/auditChain.service.ts) — SHA-256 tamper-evident per-application chain; assertChainIntact() blocks irreversible steps (disbursement) on tamper; all state changes append in the same transaction.
- **Optimistic concurrency** — version increments on every update/transition; updateMany WHERE state=existing guards transitions (409 on race); updateApplication requires version (428 if missing).

1. Borrower Onboarding & Profile

Routes: /borrowers (borrowerProfile.routes.ts, :219), /borrowers/duplicate-exceptions (:218), nested directors/shareholders/UBOs/FATCA-CRS under /borrowers (:222-225), /consent (:373).

Controllers: borrowerProfile.controller.ts, borrowerDuplicateException.controller.ts, borrowerCreditData.controller.ts, consent.controller.ts.

- **Create** POST /borrowers → borrowerProfileService.createBorrowerProfile + borrowerOnboardingService.run (idempotent). credit:create.encryptBorrowerFields() encrypts sensitive fields. Duplicate detection returns 409; admin may override (?overrideDuplicate=true, needs credit:admin).
- **Onboarding** GET/PUT /borrowers/:id/onboarding → borrowerOnboardingService; stages PROFILE/INCOME/KYC/AML/DOCUMENTS persisted to BorrowerOnboardingRun.
- **KYC/AML/credit data** POST /borrowers/:id/kyc, /aml-screening, PUT /credit-profile, /income, POST /bureau-reports → BorrowerCreditProfile, BorrowerIncome, BorrowerBureauReport, BorrowerBureauFacility. POST /borrowers/identity-check runs NRIC/SSM matching.
- **Duplicate detection** GET /borrowers/check-duplicate → checkDuplicate/checkDuplicateEnhanced. Exceptions via borrowerDuplicateException.routes.ts (POST /request, GET /pending + POST /:id/decision).
- **Consent (PDPA)** /consent → record/withdraw/check/export subject data.

Models: BorrowerProfile, BorrowerOnboardingRun, BorrowerCreditProfile, BorrowerIncome, BorrowerBureauReport, BorrowerDuplicateException, Director, Shareholder, UltimateBeneficialOwner, FatcaCrsDeclaration, RelatedPartyGroup, ConsentRecord.

2. Credit Application Creation & Submission

Routes: /applications (creditApplication.routes.ts :243) + applicationFacility.routes.ts, applicationParty.routes.ts (:244-245).

Controllers: creditApplication.controller.ts, applicationFacility.controller.ts, applicationParty.controller.ts.

- **Create** POST /applications → createApplication → CreditApplication in DRAFT; auto-assigns creating user as RM if none, defaults branch from RM, generates applicationNo, writes initial audit event, persists processing lane (lane.service.persistLane). credit:create.
- **Draft wizard** GET/PUT/DELETE /applications/draft → CreditApplicationDraft (per-user wizard state).
- **Update** PATCH /applications/:id → updateApplication — DRAFT-only for data edits; assignment-only (RM/analyst) in any non-terminal state. Mandatory version (428), optimistic concurrency (409).
- **Facilities** POST /applications/:id/facilities → ApplicationFacility (limit, tenor, rate, approving level, pricing label).
- **Parties** POST /applications/:id/parties → ApplicationParty (role, liability %, linked borrower).
- **Submit** POST /applications/:id/transition {action:'submit'} → transitionApplication. Tiered RBAC via requireTransitionPermission. **Gate:** validateSubmissionReadiness(..., 'submission') hard-gate blocks DRAFT→SUBMITTED if intake invalid; derives connected-party flag on submit (non-blocking).
- **Transitions** defined in creditApplication.service.ts:178-248. GET /:id/transitions lists valid actions; GET /:id/audit reads the trail.

Models: CreditApplication, ApplicationFacility, ApplicationParty, CreditApplicationDraft, CreditAuditEvent, BorrowerProfile, RelatedPartyGroup, Branch.

3. Credit Assessment / Underwriting

Routes (under /applications): riskAssessment, qualitativeAssessment, bureauCheck, bureauChecklist, financial/financials, industryAssessment, externalRating, rmdIssue, esg, sicr, retailIncome (+ /sme :390), profitability/walletShare/keyCounterparty/accountUtilisation (:329-332).

Controllers: matching *controller.ts.

Collects the inputs that feed scoring and the CA memo:

- **Risk** GET/POST /applications/:id/risk-assessment → RiskAssessment.
- **Qualitative** GET/PUT .../qualitative-assessment → QualitativeAssessment (management/relationship/industry/collateral scores → toFactorScores() consumed by scoring).
- **Bureau** POST .../bureau-checks → CreditBureauCheck + BureauChecklist; freshness/caps feed scoring. **Bureau adapter is a no-op placeholder** (adapters/bureau.noop.ts).
- **Financials** → FinancialStatement + FinancialLineItem + FinancialRatio (BS must balance for committee entry). SME: /sme → SmeFinancial.

- **Industry/External/RMD/ESG/SICR** → `IndustryAssessment`, `ExternalRating`, `RmdIssue`, `EsgAssessment`, `SicrAssessment` (**required for corporate apps before committee**). Retail income → `RetailIncome`. CA Memo Phase 4: profitability/wallet share/key counterparty/account utilisation.

Models: `RiskAssessment`, `QualitativeAssessment`, `CreditBureauCheck`, `BureauChecklist`, `FinancialStatement`, `FinancialLineItem`, `FinancialRatio`, `IndustryAssessment`, `ExternalRating`, `RmdIssue`, `EsgAssessment`, `SicrAssessment`, `RetailIncome`, `SmeFinancial`, `ProfitabilityLine`, `WalletShare`, `KeyCounterparty`, `AccountUtilisationSnapshot`.

4. Scoring & Risk Rating

Routes: `POST /applications/:id/score`, `GET /applications/:id/scores` (`scoring.routes.ts :264`); `/score-runs` (`:265`, override endpoint RETIRED 410 — LOS-008); `/score-overrides` (`:335`); `/scorecards`, `/scorecard-versions` (`:262-263`); `/rating-bands` (`:322`).

Controller: `scorecard.controller.ts`, `scoreOverride.controller.ts`, `ratingBandConfig.controller.ts`.

`scoringService.executeScore` (`scoring.service.ts:356`):

- 1 Resolve active `CreditScorecardVersion` (product-specific first, else generic; 409 if multiple active).
- 2 Load latest `APPROVED FinancialStatement.ratios`.
- 3 Retail: DSR (NET/GROSS) for cashflow; else DSCR/interest coverage.
- 4 Load `QualitativeAssessment` → factor scores.
- 5 Build 9 factor scores (`financial_performance`, `leverage`, `liquidity`, `cashflow`, `management`, `industry`, `collateral`, `relationship`, `market_conditions`).
- 6 Apply **missing-data policy** for absent factors; collect `missingInputs`.
- 7 **Governance warnings** via `scoreFactorDefinitionService.validateFactorWeights`.
- 8 Weighted `totalScore` → risk rating via configurable `RatingBandConfig` (DB active bands; fallback `mapTotalScoreToRiskRating`).
- 9 **Bureau caps** (`applyBureauCaps`, caps only worsen) + freshness flags.
- 10 Create `CreditScoreRun` (with `ratingBandVersion` + `policyVersion` provenance), persist rating, append `SCORE_RUN_CREATED` audit — all atomic.

Override: `direct overrideScore` rejects ≥ 2 -notch changes; material overrides go through `/score-overrides` (dual-approval, SOD enforced, rate-limited 5/min) → `ScoreOverrideApproval`.

Committee freshness gate: ≥ 1 score run AND latest run not stale vs latest material update; absolute ceiling `config.credit.scoreMaxAgeDays` (default 30d).

5. Committee & Approval

Routes: `approval.routes.ts (:254)` — `/approval-matrices` CRUD + lookup, `POST /applications/:id/approvals`, `GET .../approval-matrix-applicability`, `GET .../approvals`; `committee.routes.ts (/committee, :268)`; `signoff.routes.ts (:326)`; `creditRecommendation.routes.ts (:295)`.

Controllers: `approval.controller.ts`, `committee.controller.ts`, `signoff.controller.ts`, `creditRecommendation.controller.ts`.

- **Authority lookup** `POST /approval-matrices/lookup` → `lookupApprovalAuthority` (`exposure`, `rating`, `branch`, `lane`) → `CreditApprovalMatrix` (`active`, `effective`, `min/max exposure`, `rating`)

ordinal; branch rows take precedence; lane override PERSONAL_FAST/SME ⇒ requiredApproverCount=2).

- **Board-band trigger** (approvalAction.service.ts:45): exposure ≥ RM5M or rating ≥ CC ⇒ committee/board-level authority.
- **Submit approval** POST /applications/:id/approvals (credit:approve + enforceCreditSOD()) → submitApprovalAction. Only in COMMITTEE_REVIEW (APPROVAL_ELIGIBLE_STATES, LOS-001). Enforces SOD (RM-self, maker-checker), conflict-of-interest (approver ≠ signoff signatory), authority hierarchy, duplicate-check; records CreditDecision; auto-creates conditions for CONDITIONAL; advances state only when approvalsCollected ≥ requiredApproverCount; routes next approver.
- **Committee** committee.service.ts: meeting CRUD, members/attendance, **quorum** (present ≥ quorumMin + risk-member check), agenda items, votes (ABSENT can't vote, member must match user), finalizeDecision (majority APPROVE/REJECT/DEFER; REJECT needs ≥10-char comment; transitions linked app with skipApprovalChainCheck). Gated by enforceCommitteeSOD().
- **CA memo sign-off** ApplicationSignoff (PREPARED_BY/REVIEWED_BY/CONCURRED_BY). **Committee entry gate** requires all three signed by distinct users (SOD) — enforced in transitionApplication + committeeEntryGate.ts (freezes assessment + locks memo last, irreversible).
- **Recommendation SOD:** recommendation author cannot be final decision actor (server-side on approve/reject).
- **Approval-chain gate:** transitionApplication blocks approve/reject from COMMITTEE_REVIEW unless requiredApproverCount distinct APPROVE decisions collected (unless from committee finalize).

Models: CreditApprovalMatrix (+CreditApprovalMatrixVersion), CreditDecision, CommitteeMeeting, CommitteeMember, CommitteeAgendaItem, CommitteeVote, ApplicationSignoff, CreditRecommendation, ApplicationAssessmentResult.

6. CA Memo & Document Lifecycle

Routes: GET /applications/:appId/ca-memo/preview, /ca-memo, /approval-pack, POST/GET /applications/:appId/ca-memo-versions (+ /latest, /locked, /:versionNumber, /lock, /unlock) (:301-314); creditDocument.routes.ts (:233).

Controllers: caMemoPdf.controller.ts, creditMemoVersion.controller.ts, approvalPack.controller.ts, creditDocument.controller.ts.

- **CA Memo** generateCaMemo/previewCaMemo → caMemoPdf.service.getCaMemoData + buildHtml (S1–S7 sections: loan request/facilities, borrower, financials, risk score, bureau & compliance, collateral & guarantees, decision/sign-off/conditions). preview returns locked snapshot if one exists, else regenerates live.
- **Memo versioning** CreditMemoVersion — immutable snapshots, versionNumber, isLocked. Locking prevents regeneration (409); unlock is credit:admin. Locked version used by generateCaMemo.
- **Approval pack** getApprovalPack → approvalPack.service → consolidated pack for committee.
- **Documents** creditDocument.service.ts: upload, list/get/replace/versions, SHA-256 hash + verify, AV-scan status (PATCH /credit-documents/:id/av-status — service-API-key route above the auth gate), verify/reject, presigned download, requirements/checklist. assertCreditDocumentAccess/assertApplicationDocumentAccess gate access. AV-clean required before verification.
- **E-sign readiness** GET /applications/:id/esign-readiness requires a VERIFIED LETTER_OF_OFFER.

Models: CreditDocument (+CreditDocumentVersion), CreditMemoVersion, DocumentRequirement, ApplicationAssessmentResult.

7. Conditions, Offer & Acceptance

Routes: condition.routes.ts (/applications/:applicationId/conditions, cp-completion) + conditionItem.routes.ts (/conditions, :279); loo.routes.ts (/applications/:appId/loo/*, :353); rejection.routes.ts (:356).

Controllers: condition.controller.ts, loo.controller.ts, rejection.controller.ts.

- **Conditions** condition.service.ts: list/create/update/complete/waive/checkCpCompletion → Condition (PRECEDENT/SUBSEQUENT; PENDING/COMPLETED/WAIVED/EXPIRED; isFulfilled, waiverReason). Can be created against a CONDITIONAL decision.
- **CP fulfilment gate:** make_offer from CONDITION_FULFILMENT blocks if any PRECEDENT unfulfilled & not WAIVED. Same gate in disburseOrder.
- **LOO** loo.service.generate — only APPROVED/OFFER; renders templates/loo.html, enqueues async PDF, saves CreditDocument classification LETTER_OF_OFFER, sets OFFER + looGeneratedAt/looExpiryDate/looVersion. regenerate, getStatus, checkExpiry, checkAndNotifyExpiring (daily job; warnings T-7/3/1, marks expired).
- **Acceptance** accept_offer requires a VERIFIED LETTER_OF_OFFER + not expired. decline_offer (credit:approve) → REJECTED.
- **Rejection** rejection.service.ts: notifyRejection, copyToNewApplication (clone REJECTED → DRAFT; copies parties/facilities, not decisions/documents/conditions; sets parentApplicationId).

Models: Condition, CreditDocument (LOO), CreditDecision, LetterOfAcceptance.

8. Disbursement

Routes: disbursement.routes.ts (under /applications, :347): POST/:appId/disbursement, GET, POST /approve, POST /disburse, POST /cancel, GET /readiness.

Controller: disbursement.controller.ts → disbursement.service.ts.

- **Readiness** checkDisbursementReadiness: app in ACCEPTED; no pending PRECEDENT conditions; ≥1 APPROVE CreditDecision; signed LOO VERIFIED.
- **Create order** createOrder — **fails closed if CBS adapter is a placeholder** (assertRecordOnlyAllowed('cbs')); verifies audit chain BEFORE money moves; amount ≤ sum of approved facilities; no existing non-cancelled order; generates DO-YYYY-NNNNN; creates DisbursementOrder + audit event.
- **Approve** approveOrder — maker-checker (approver ≠ requester).
- **Disburse** disburseOrder — three-role segregation (disburser ≠ approver ≠ requester); re-checks precedent conditions; atomically sets order DISBURSED + app → DISBURSED + audit.
- **Cancel** cancelOrder — reason required; can't cancel DISBURSED.
- Direct ACCEPTED→DISBURSED transition is **blocked**; disburse action redirects to the order workflow (creditApplication.service.ts:1450-1470).

Models: DisbursementOrder, CreditApplication, Condition, CreditDecision, CreditDocument.

9. Monitoring & Early Warning

Routes: monitoring.routes.ts (under /applications, :288) + monitoringItem.routes.ts (:289). Jobs: collateralInsuranceMonitor.job.ts, amlRescreenChecker.ts, monitor.job.ts.

Controller: monitoring.controller.ts → monitoring.service.ts.

- **Facility health** GET/POST/PATCH .../health → FacilityHealth (healthStatus, review dates, frequency).
- **Covenants** → CovenantDefinition + CovenantTest; non-compliant test auto-creates EarlyWarningSignal (FINANCIAL_RATIO=CRITICAL, DSCR/LTV=HIGH).
- **Payments** → PaymentEvent; updatePaymentStatus triggers overdue check (LATE_90/MISSED → HIGH PAYMENT_OVERDUE).
- **Early warning signals** GET .../signals, create/resolve → EarlyWarningSignal.
- **Collateral/insurance monitor** (daily job): flags stale collateral valuations (≥9m MEDIUM, ≥12m HIGH) and expiring insurance; hasStaleCollateralValuations() is a **hard-block** in transitionApplication for activate/disburse (>12 months). Configurable collateral.valuation.warning_months/block_months.
- **AML re-screen** (quarterly): amlRescreenChecker.ts → PEP/sanctions re-screen for active borrowers → AmlRescreenEvent (adapter is placeholder).

Models: FacilityHealth, CovenantDefinition, CovenantTest, PaymentEvent, EarlyWarningSignal, Collateral/CollateralValuation, InsuranceCover, AmlRescreenEvent.

10. Reports & Dashboard

Routes: dashboard.routes.ts (/dashboard, :282): /my-work, /pipeline, /approval-inbox, /exposure, /committee-calendar, /exposure-summary, /work-queue, /alerts, /activity, /team-performance. reports.routes.ts (/reports, :285): /pipeline, /exposure, /approval-turnaround (CSV/XLSX/JSON).

Controller: dashboard.controller.ts → dashboard.service.ts.

- **Dashboard** methods (dashboard.service.ts:449-1603): pipeline, my-work, approval-inbox (urgency grouping), exposure, exposure-summary, committee-calendar, work-queue (6 buckets + SLA compliance %), alerts (High DSR / Expired Bureau / AML Review), activity feed, team-performance (credit:admin). All read-only.
- **Reports** reuse dashboard methods + CSV/XLSX export utils; each export logged via exportAudit.service.logCreditExport → CreditExportEvent. creditExportLimiter rate limit.

11. SLA & Escalation

Routes: creditSla.routes.ts (/sla, :341): /policies CRUD (admin), /breaches, /breaches/:applicationId, /breaches/:id/acknowledge (credit:write), /breaches/:id/resolve (credit:approve), /check (manual trigger).

Controller: creditSla.controller.ts → creditSla.service.ts.

- **Policy** → CreditSlaPolicy (targetState, slaHours, notifyRoles, escalateAfterHours/escalateToState, productType, branch override via CreditSlaPolicyBranchOverride).
- **Due-date** getEffectiveSlaForApplication/computeSlaDueDate — earliest active policy matching state+product, honoring branch overrides.
- **Breach** checkAndRecordBreaches — 15-min cron (creditSlaChecker.ts, env CREDIT_SLA_CRON) → CreditSlaBreach + SLA_BREACH audit.

- **Escalation** processEscalations — after escalateAfterHours, auto-advances app to escalateToState, SLA_ESCALATION audit.
- **Management** acknowledgeBreach/resolveBreach; dashboard consumption.

Models: CreditSlaPolicy, CreditSlaPolicyBranchOverride, CreditSlaBreach, CreditApplication, CreditAuditEvent.

12. AI-Assisted Credit

Routes: creditAi.routes.ts (under /applications, :367): /ai/duplicates (A6), /ai/red-flags (A5), /ai/narrative (A4), /ai/compliance (A13), /ai/exceptions (A15), /ai/interactions, /ai/overrides (credit:write).

Controllers: creditAi.controller.ts → credit-ai.service.ts, creditAiCompliance.service.ts, creditRedFlag.service.ts, creditNarrative.service.ts, creditAutoException.service.ts, creditDuplicate.service.ts.

- callAi<T>() (credit-ai.service.ts:56): loads active AiPromptVersion (feature-scoped, versioned), calls OpenAI (gpt-4o / gpt-4o-mini), JSON-object response, records AiInteraction (entity, user, input hash, tokens, latency, cost USD). 503 if OPENAI_API_KEY unconfigured.
- **Compliance (A13)** runAiComplianceCheck — feeds AML tier/sanctions, doc verification, bureau hits, open conditions; surfaces soft concerns (deterministic failures handled separately).
- **Human override governance** recordOverride → AiOverride (links to interaction, reason, original vs overridden). Audit via GET /ai/interactions.

Models: AiPromptVersion, AiInteraction, AiOverride.

13. DLP / Export Control / Audit

Routes: dlp.routes.ts (mounted router.use('/', dlpRoutes) :344): POST /export-tokens, GET /exports/pipeline, GET /exports/exposure (all require requireExportToken + creditExportLimiter). Audit via auditChain.service.ts, exportAudit.service.ts, piiReadLog.middleware.ts.

- **Export tokens** dlpService.createExportToken/consumeExportToken — Redis-backed, 5-min TTL, single-use, prevents link sharing/replay.
- **PII redaction** redactPiiPatterns (NRIC/phone/email/bank account); redactObject (ALWAYS_REDACT_FIELDS for non-admin). Admin sees only pattern-redaction.
- **Watermarking** injectCsvWatermark/injectJsonWatermark — user ID + timestamp + confidentiality warning.
- **Export audit** logCreditExport → CreditExportEvent (user, reportType, format, filters, rowCount, ip, userAgent).
- **Audit chain** immutable SHA-256 chain per application; auditRetention.job.ts handles retention.
- **PII read log** piiReadLog.middleware.ts → PiiReadLog; GET /borrowers/:id/contact-nric/reveal (credit:write) is PII-logged.
- **Field encryption** fieldEncryption.middleware.ts/encryption.service.ts encrypts sensitive borrower fields at rest; decrypted on borrower GETs.

Models: CreditExportEvent, PiiReadLog, CreditAuditEvent, BorrowerProfile (encrypted fields), FeatureFlag.

Cross-flow notes for maintainers

- **Single state machine** drives the whole LOS: `creditApplication.service.ts:178-248` is canonical; every gate (submission readiness, committee entry, SOD, approval chain, collateral freshness, CP fulfilment, LOO, disbursement) is enforced inside `transitionApplication / approvalActionService / disbursement.service`, not in routes.
- **Feature-flag gating:** module-level `credit:module` required for everything; flag admin mutations require MFA.
- **SOD is layered:** route middleware + service checks + committee gates.
- **Atomicity:** state writes + decision/score/order records + audit-chain append commit in the same `$transaction`, guarded by optimistic version and `pg_advisory_xact_lock`.
- **Adapters are fail-closed placeholders** (`bureau.noop.ts`, `aml.placeholder.ts`, `cbs.placeholder.ts`, `esign.placeholder.ts`, `ocr.placeholder.ts`): `assertRecordOnlyAllowed('cbs')` blocks disbursement until live.
- **Processing lanes** (P2-2): `lane.service` persists lane on create; lane affects approval depth (`PERSONAL_FAST/SME` $\Rightarrow$ 2 approvers) and missing-data policy resolution.